Steganography

Michael Effinger, Zachary Kirtz

The construction

Our Idea

We are two guys with a dream. This was to embed a secret message of sorts inside of a executable of sorts, This required 2 things, One we build what one could call a transpiler, two we build a parser for the dubious binary that has been constructed.

Guide to linking(CS 4175 LINK)

Has this ever been you(I am talking to you (pick out student to single out)).

```
Ld /Users/cschmitz/Library/Developer/Xcode/DerivedData/hello-world-hcfkudnckbodjzakktmdddluzagj/Build/Products/Debug/hello-world normal x86_64 (in target: hello-world)
    cd /Users/cschmitz/Development/CS\ Dev/Explorations/cpp/hello-world/hello-world
    export MACOSX_DEPLOYMENT_TARGET=10.14
    /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin/clang++ -arch x86_64 -isysroot /Applications/Xcode.app/Contents/Developer/
  Platforms/MacOSX.platform/Developer/SDKs/MacOSX10.14.sdk -L/Users/cschmitz/Library/Developer/Xcode/DerivedData/hello-world-hcfkudnckbodjzakktmdddluzagj/Build/Products/
  Debug -F/Users/cschmitz/Library/Developer/Xcode/DerivedData/hello-world-hcfkudnckbodjzakktmdddluzagj/Build/Products/Debug -filelist /Users/cschmitz/Library/Developer/
  Xcode/DerivedData/hello-world-hcfkudnckbodjzakktmdddluzagj/Build/Intermediates.noindex/hello-world.build/Debug/hello-world.build/Objects-normal/x86_64/hello-
  world.LinkFileList -mmacosx-version-min=10.14 -Xlinker -object_path_lto -Xlinker /Users/cschmitz/Library/Developer/Xcode/DerivedData/hello-world-
  hcfkudnckbodjzakktmdddluzagj/Build/Intermediates.noindex/hello-world.build/Debug/hello-world.build/Objects-normal/x86_64/hello-world_lto.o -Xlinker -export_dynamic -
  Xlinker -no_deduplicate -stdlib=libc++ -Xlinker -dependency_info -Xlinker /Users/cschmitz/Library/Developer/Xcode/DerivedData/hello-world-hcfkudnckbodjzakktmdddluzagj/
  Build/Intermediates.noindex/hello-world.build/Debug/hello-world.build/Objects-normal/x86_64/hello-world_dependency_info.dat -o /Users/cschmitz/Library/Developer/Xcode/
  DerivedData/hello-world-hcfkudnckbodjzakktmdddluzagj/Build/Products/Debug/hello-world
```

```
Undefined symbols for architecture x86_64:
  "Log::Info(char const*)", referenced from:
      _main in main.o
  "Log::Warn(char const*)", referenced from:
      _main in main.o
  "Log::Error(char const*)", referenced from:
      _main in main.o
  "Log::SetLevel(Log::Level)", referenced from:
      _main in main.o
ld: symbol(s) not found for architecture x86_64
clang: error: linker command failed with exit code 1 (use -v to see invocation)
```

symbol(s) not found for architecture x86_64

linker command failed with exit code 1 (use -v to see invocation)

Activity Log Complete 1/23/19, 8:37 AM 0.4 seconds

1 error

How is this possible

The idea is that we hid the obfuscated message, in the symbol the linker uses. So we used linux ELF format(executable linker format). Basically when the compiler turns all of your code into Assembly, it must go around and find all of the forward declarations, and link them with their definitions. We must tap into this. Because function names are stored as strings(text) we can use these to hide a message!!!!!

The method

So what happens is, you give the transpiler a message, and converts it into some innocuous c++ code(if you don't happen to ELF). It maps each letter of your message into a component of some symbol(i.e class, variable, method name).

```
constexpr std::array<char const * const,16> prefix{
    [0]="_unsignedint_",
    [1]="_int_",
    [2]="_unsignedshort_",
    [3]="_short_",
    [4]="_long_",
    [5]="_unsignedlong_",
    [6]="_longlong_",
    [7]="_unsignedlonglong_",
    [8]="_float_",
    [9]="_double_",
    [10]="_longdouble_",
    [11]="_uint8_",
    [12]="_int8_",
    [13]="_uint16_",
    [14]="_int16_",
    [15]="_char_",
};
```

Biggest difficulty with creation

The number one difficulty with writing this file, is preserving its compilability, It's not too hard to generate a string based on a variety of keywords, the hard part is instead trying to insert these words into a premade c++ file with minimum friction. The system requires the creation of the strings, followed by their seamless insertion into a premade cpp outline.

How is this sneaky

Well on the surface level, its not at all. If we look straight at the cpp file, you can tell no one and their mothers would ever write code this obtuse(‘cept for the c++ standard lib guys). The trick is though, that we wouldn't ever give out this cpp file. Instead we give out the binary. Much like how that C guy did it with his compiler

```
class _unsignedlonglong_ultimateUSstandardTRANSLATORwithNewFunkyMode {

    private:

    float _unsignedlonglong_amazingUSstandardTRANSLATORFinalEdition;

    void _longlong_coolestusSTANDARDTRANSLATORandKnuckles(float input){

    }

    float _unsignedlonglong_amazingUSSCIENTIFICTranslatorUltimateEdition(){
        return _unsignedlonglong_amazingUSstandardTRANSLATORFinalEdition;
    }

    void _unsignedlonglong_radusSCIENTIFICTRANSLATORAdvanced(float input){
        _longlong_coolestusSTANDARDTRANSLATORandKnuckles(input: input /3.28);
    }

    void _longlong_crazyUSMTCTRANSLATORandKnuckles(float input){
        _longlong_coolestusSTANDARDTRANSLATORandKnuckles(input: input * 3.28);
    }

    friend class _unsignedlonglong_specialusMtcTRANSLATORFinalEdition;


};


class _unsignedlonglong_specialusMtcTRANSLATORFinalEdition{
    _unsignedlonglong_ultimateUSstandardTRANSLATORwithNewFunkyMode _m_S_unsignedlonglong_specialusMtcT

    _unsignedlonglong_ultimateUSstandardTRANSLATORwithNewFunkyMode _get_S_unsignedlonglong_specialusMt
        return _m_S_unsignedlonglong_specialusMtcTRANSLATORFinalEdition;
    }

};
```

The trick

So the only way for a predator to actually see the scrambled message, is if they use a tool to actually peek inside the binary and see what symbols are defined via the ELF format. We also want to make it seem as if its a real program. Which is why we modified our imperial to metric c++ code from Computer Science 1. So now it would simply look like a pretty crappy first time programers piece of code.

```
                 U _ZNSt7__cxx1118basic_stringstreamIcSt11char_traitsIcESaIcEEaSEOS4_@GLIB
                 U _ZNSt7__cxx1118basic_stringstreamIcSt11char_traitsIcESaIcEEC1ERKNS_12ba
                 U _ZNSt7__cxx1118basic_stringstreamIcSt11char_traitsIcESaIcEEC1Ev@GLIBCXX
                 U _ZNSt7__cxx1118basic_stringstreamIcSt11char_traitsIcESaIcEED1Ev@GLIBCXX
0000000000007ad6 W _ZNSt8__detail10_Hash_nodeISt4pairIKNSt7__cxx1112basic_stringIcSt11char
0000000000007ad6 W _ZNSt8__detail10_Hash_nodeISt4pairIKNSt7__cxx1112basic_stringIcSt11char
000000000000873e W _ZNSt8__detail14_Node_iteratorISt4pairIKNSt7__cxx1112basic_stringIcSt11
000000000000873e W _ZNSt8__detail14_Node_iteratorISt4pairIKNSt7__cxx1112basic_stringIcSt11
000000000000515c W _ZNSt8__detail15_Hash_code_baseINSt7__cxx1112basic_stringIcSt11char_tra
d_hashELb1EEC1Ev
000000000000515c W _ZNSt8__detail15_Hash_code_baseINSt7__cxx1112basic_stringIcSt11char_tra
d_hashELb1EEC2Ev
0000000000004fda W _ZNSt8__detail15_Hash_node_baseC1Ev
0000000000004fda W _ZNSt8__detail15_Hash_node_baseC2Ev
0000000000005168 W _ZNSt8__detail15_Hashtable_baseINSt7__cxx1112basic_stringIcSt11char_tra
20_Default_ranged_hashENS_17_Hashtable_traitsILb1ELb0ELb1EEEEC1Ev
0000000000005168 W _ZNSt8__detail15_Hashtable_baseINSt7__cxx1112basic_stringIcSt11char_tra
20_Default_ranged_hashENS_17_Hashtable_traitsILb1ELb0ELb1EEEEC2Ev
0000000000007af2 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
000000000000813e W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
tSt5tupleIJOS8_EESI_IJEEEEEPSB_DpOT_
0000000000007672 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
0000000000006a48 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
00000000000092f6 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
0000000000006064 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
0000000000007702 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
Em
0000000000007680 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
00000000000051a8 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
00000000000051a8 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
00000000000051c6 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
00000000000051c6 W _ZNSt8__detail16_Hashtable_allocISaINS_10_Hash_nodeISt4pairIKNSt7__cxx1
0000000000008444 W _ZNSt8__detail17_RehashStateGuardINS_20_Prime_rehash_policyEEC1ERS1_
0000000000008444 W _ZNSt8__detail17_RehashStateGuardINS_20_Prime_rehash_policyEEC2ERS1_
000000000000847e W _ZNSt8__detail17_RehashStateGuardINS_20_Prime_rehash_policyEED1Ev
000000000000847e W _ZNSt8__detail17_RehashStateGuardINS_20_Prime_rehash_policyEED2Ev
0000000000008f68 W _ZNSt8__detail19_Node_iterator_baseISt4pairIKNSt7__cxx1112basic_stringI
0000000000008f68 W _ZNSt8__detail19_Node_iterator_baseISt4pairIKNSt7__cxx1112basic_stringI
000000000000504e W _ZNSt8__detail20_Prime_rehash_policy8_M_resetEm
0000000000005012 W _ZNSt8__detail20_Prime_rehash_policyC1Ef
0000000000005012 W _ZNSt8__detail20_Prime_rehash_policyC2Ef
0000000000005184 W _ZNSt8__detail21_Hashtable_ebo_helperISaINS_10_Hash_nodeISt4pairIKNSt7__
0000000000005184 W _ZNSt8__detail21_Hashtable_ebo_helperISaINS_10_Hash_nodeISt4pairIKNSt7__
0000000000009251 W _ZNSt8__detail22_Hashtable_hash_traitsISt4hashINSt7__cxx1112basic_strin
00000000000064d0 W _ZNSt8__detail9_Map_baseINSt7__cxx1112basic_stringIcSt11char_traitsIcES
20_Default_ranged_hashENS_20_Prime_rehash_policyENS_17_Hashtable_traitsILb1ELb0ELb1EEELb1E
                 U _Znwm@GLIBCXX_3.4
0000000000004d6a W _ZnwmPv
```

An issue arises

Now that we have our message in an incomprehensible state, We have nothing to worry about, it seems as though getting this data would be impossible, but now we have an issue, how do we get the data. Luckily we have Zach.

A Very Vexing Parse

Zach, using his extraordinary brain, will attempt to do the unthinkable, parse the unparsable, and make the whole world bend to his wills. The main hiccup with turning these secret messages back to normal, is a little thing called Itanium, One must not only create a general parser for these messages, but also deal with all the headaches of the itanium ABI

Biggest difficulty with parsing

A big issue with the parsing, is even with our careful consideration, variable names still will not come out unmangled. This means that understanding this mangling is a pivotal part of working with ELF. The Linker uses C as its lingua franca, this means that no matter what language you are using, there will always be a way to interface with c. however, this causes some problems because as you all know, We like to work in c++, and C lacks many of the things I love, such as classes member functions and namespaces. So Itanium has to get creative, by mangling the function names.

Itanium Mangling

While at first these symbols seem to be impossible to reason about, Once you dwell in the depths like we have, it becomes very obtainable. Firstly every mangled token starts with an _Z. Namespaces too, get mangled. When looking at a function like int math::add(int, double), we get a much nastier name with _ZN4math3addEid. You can see now that while its not exactly nice to read, it isn't very complex.

Conclusion

Once the names are all demangled,ordering becomes the last thing we must work with. This can be handled relatively simply though with just a standard ordering that we defined. After all of this, its relatively smooth sailing, and that is how we used Elf and Itanium along with some c++ helpers along the way, to hide a secret message.